CMSC131: Methods, Scope, and Parameters

Chapter 5

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Methods? 3

2 The Parts of a Method Declaration 3

2.1 Method header versus method body 4

3 Calling a Method: What Actually Happens? 4

4 Parameters and Arguments 5

5 Returning a Value versus Printing a Value 5

6 Static Methods and Non-Static Methods 6

6.1 Static methods belong to the class 6

6.2 Instance methods belong to objects 7

6.3 Why can't static main directly use an instance method? 7

7 Local Variables, Parameters, and Scope 8

7.1 Shadowing a field 8

8 Java Passes Arguments by Value 9

8.1 Primitive example: changing the parameter does not change the caller 9

8.2 Object reference example: the same object can be changed 9

8.3 Reassigning a parameter does not replace the caller's reference 10

8.4 What about immutable Strings? 10

9 Method Overloading 10

10 Methods with Conditions and Loops **11**

10.1 Example: count the digits of a positive integer 11

10.2 Example: determine whether a number is prime 11

11 Multiple Calls Create Separate Invocations **12**

12 Method Names, Documentation, and Good Design **12**

13 A Complete Program Built from Several Methods **13**

14 Reading and Debugging Method Calls **14**

14.1 Frequent syntax problems and what they mean 14

15 Quick Reference: What to Look For in a Method **15**

1 Why Do We Need Methods?

Until now, our Java programs have mostly placed instructions directly inside `main`. This works for small programs, but imagine writing a program that calculates the area of a rectangle ten different times. Copying the same multiplication into ten places makes the program longer and creates ten places where we could accidentally introduce a mistake. A **method** gives a sequence of instructions a name. We can then ask Java to execute those instructions whenever we need them.

A method is useful for more than avoiding repetition. It allows us to divide a large task into smaller, understandable tasks. For example, a program might have separate methods to read a score, determine a letter grade, and print a report. Each method has one clear job. We can test each job separately, and we can change its implementation without rewriting the rest of the program.

```
public class Greetings {
    public static void main(String[] args) {
        sayHello();
        sayHello();
        sayHello();
    }

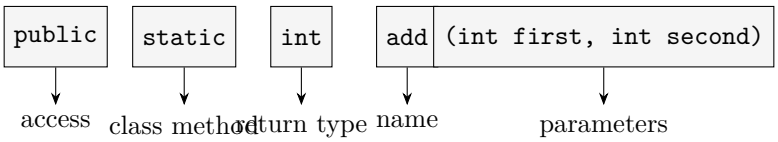
    public static void sayHello() {
        System.out.println("Hello from CMSC131!");
    }
}
```

This prints the same greeting three times. We *defined* the method once, but *called* it three times. A definition describes what a method will do; a call actually causes its body to run. Java does not execute the body merely because we wrote the definition in the class.

2 The Parts of a Method Declaration

Consider the following method. Every piece of its first line has a specific purpose.

```
public static int add(int first, int second) {
    int answer = first + second;
    return answer;
}
```



`public` is an access modifier: it allows code from other classes to call this method, subject to normal Java access rules. `static` means this method belongs to the class rather than requiring a particular object. `int` is the return type: callers will receive an integer result. `add` is the method name. The parentheses hold the **parameters**, the inputs the method receives. The braces enclose the **method body**, which contains the statements to execute.

2.1 Method header versus method body

The method **header** says how other code can call the method; the **body** says what happens when it is called. Parameters are declared in the header, but their values are supplied at each call. You cannot write a method definition directly inside another method in Java: methods are declared in a class (or another allowed type declaration), not nested inside `main`.

```
public class Calculator {
    public static void main(String[] args) {
        int total = add(4, 7);
        System.out.println(total); // 11
    }

    public static int add(int x, int y) {
        return x + y;
    }
}
```

Notice that `add` is outside `main`, but still inside the `Calculator` class. Also notice that `main` appears before `add`; Java does not require methods to be declared in the order in which they are called.

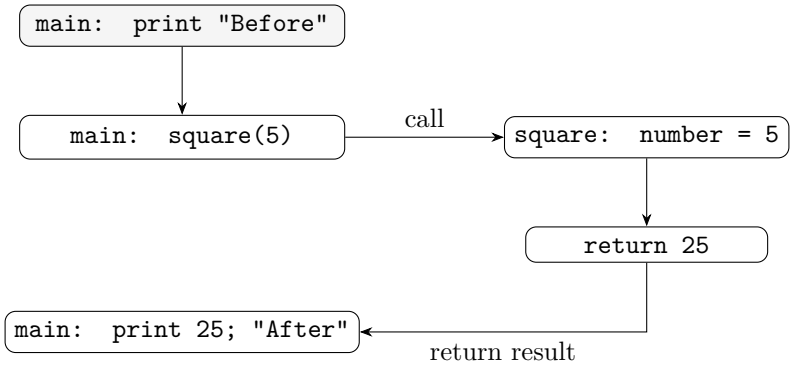
3 Calling a Method: What Actually Happens?

A method call temporarily transfers control from the calling method to the called method. Java evaluates the arguments, creates a new method invocation with its own parameters and local variables, runs its body, and then returns control to the statement after the call. If the method produces a value, that value becomes the result of the call expression.

```
public static int square(int number) {
    System.out.println("Inside square");
    return number * number;
}

public static void main(String[] args) {
    System.out.println("Before");
    int result = square(5);
    System.out.println(result);
    System.out.println("After");
}
```

The output is `Before`, then `Inside square`, then `25`, then `After`, each on a separate line. When Java reaches `square(5)`, it does not skip over the method or execute the remaining lines of `main` immediately. It finishes that call first.



4 Parameters and Arguments

A **parameter** is the variable declared in a method's header. An **argument** is the expression supplied at the call site. In the next example, `length` and `width` are parameters, whereas `8` and `3` are arguments.

```
public static int rectangleArea(int length, int width) {
    return length * width;
}

int area = rectangleArea(8, 3); // area becomes 24
```

The call must supply the required number of arguments, and each argument must be compatible with the corresponding parameter type. Java matches them by **position**, not by the names of the variables.

```
public static void showDifference(int a, int b) {
    System.out.println(a - b);
}

showDifference(10, 4); // prints 6
showDifference(4, 10); // prints -6
```

Changing the order changes which value becomes `a` and which becomes `b`. The caller's variable names do not need to match the parameter names.

```
int firstNumber = 12;
int secondNumber = 5;
showDifference(firstNumber, secondNumber); // prints 7
```

5 Returning a Value versus Printing a Value

A method that **prints** a number displays it on the screen. A method that **returns** a number gives it back to the caller so the caller can store it, compare it, print it, or use it in a larger calculation. These are different actions.

```
public static void printDouble(int x) {
    System.out.println(x * 2);
}

public static int getDouble(int x) {
```

```
    return x * 2;
}

printDouble(7); // prints 14
int answer = getDouble(7); // stores 14; does not print
System.out.println(answer + 1); // prints 15
```

A `void` method does not return a value. It can still perform useful work, such as printing a message or changing the state of an object. A non-`void` method must return a compatible value along every reachable path that completes normally.

```
public static int larger(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

Both possible paths return an `int`. Contrast this with an invalid definition:

```
// Does not compile: missing return when x <= 0.
public static int badExample(int x) {
    if (x > 0) {
        return x;
    }
}
```

A `return` statement ends the current method call immediately. Statements after an unconditional `return` in the same block are unreachable and cause a compile-time error.

```
public static boolean isPositive(int number) {
    return number > 0;
    // System.out.println("Done"); // unreachable
}
```

A `void` method may use `return;` with no value to finish early:

```
public static void printIfPositive(int x) {
    if (x <= 0) {
        return;
    }
    System.out.println(x);
}
```

6 Static Methods and Non-Static Methods

This distinction becomes especially important when we start working with classes, objects, and constructors. For now, focus on *who owns the method and how we call it.*

6.1 Static methods belong to the class

A `static` method is associated with the class itself. In beginner programs, we commonly call a static method from `main`, which is also static. Within the same class, we can simply write its name. From

another class, we can write `ClassName.methodName(...)` if it is accessible.

```
public class MathHelper {
    public static int triple(int n) {
        return 3 * n;
    }
}

// In another class:
int result = MathHelper.triple(4); // 12
```

Java's library provides familiar examples such as `Math.sqrt(25)` and `Math.max(3, 9)`. These methods do not require us to create a `Math` object.

6.2 Instance methods belong to objects

A non-static method is also called an **instance method**. It runs on a particular object, and inside the method `this` refers to that object. Consider two counters that each maintain their own value.

```
public class Counter {
    private int count;

    public Counter(int startingValue) {
        count = startingValue;
    }

    public void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

```
Counter first = new Counter(0);
Counter second = new Counter(10);
first.increment();
first.increment();
second.increment();
System.out.println(first.getCount()); // 2
System.out.println(second.getCount()); // 11
```

Each object has its own `count`. The constructor `Counter(int startingValue)` runs when `new Counter(...)` creates an object and establishes its initial state. A constructor is *not* an ordinary method: it has no return type, and its name must match the class. We will study constructors and object memory in more depth in the next chapter.

6.3 Why can't static main directly use an instance method?

`main` is static, so it runs without a particular `Counter` object. A call such as `increment()` has no target object in that context. We first create or obtain a Counter object, then call `first.increment()`. In contrast, `MathHelper.triple(4)` names the class directly because `triple` is static.

	Static method	Instance method
Associated with	Class	Particular object
Typical call	`Math.max(2,5)`	`first.increment()`
Can directly use `this`?	No	Yes
Direct access to instance fields?	Needs an object reference	Yes, on the current object

7 Local Variables, Parameters, and Scope

Scope describes the region of code where a name can be used. A variable declared inside a method is generally a **local variable**; it belongs to that invocation, not to every other method in the class. A parameter is also local to its method invocation.

```
public static void greet(String name) {
    String message = "Hello, " + name;
    System.out.println(message);
}

public static void main(String[] args) {
    greet("Ada");
    // System.out.println(message); // compile error
    // System.out.println(name); // compile error
}
```

The variables `name` and `message` do not exist as accessible names inside `main`. The method `greet` has its own local context. Similarly, a variable declared inside a block is unavailable after that block ends.

```
if (true) {
    int points = 10;
    System.out.println(points); // 10
}
// System.out.println(points); // compile error
```

A `for` loop's variable declared in its initializer is normally limited to that loop:

```
for (int i = 0; i < 3; i++) {
    System.out.println(i);
}
// System.out.println(i); // compile error
```

7.1 Shadowing a field

A parameter or local variable can have the same name as a field, but that can make code confusing. Within an instance method, `this.fieldName` explicitly refers to the current object's field.

```
public class Student {
    private String name;

    public Student(String name) {
        this.name = name;
    }

    public String getName() {
```

```
        return name;
    }
}
```

Here `this.name` is the field, while the plain `name` in the constructor refers to the parameter. Without `this`, writing `name = name;` would assign the parameter to itself, leaving the field unchanged.

8 Java Passes Arguments by Value

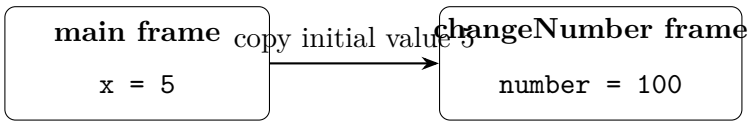
Java is **always pass-by-value**. This phrase is easy to misunderstand because primitive values and object references behave differently. For a primitive argument, Java copies the primitive value into the parameter. For an object argument, Java copies the *reference value* into the parameter. It never allows a method to replace the caller's variable directly just by assigning to the parameter.

8.1 Primitive example: changing the parameter does not change the caller

```
public static void changeNumber(int number) {
    number = 100;
    System.out.println("Inside: " + number);
}

public static void main(String[] args) {
    int x = 5;
    changeNumber(x);
    System.out.println("Outside: " + x);
}
```

The method prints `Inside: 100`, but the caller prints `Outside: 5`. When `changeNumber(x)` is called, Java copies 5 into a separate parameter variable named `number`. Assigning 100 to that parameter changes only the copy.

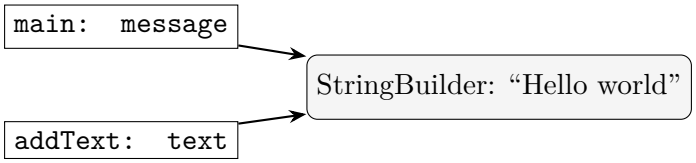


8.2 Object reference example: the same object can be changed

When an object reference is passed, both the caller's variable and the parameter can refer to the same object. If the method calls a mutating operation on that object, the caller can observe the change.

```
public static void addText(StringBuilder text) {
    text.append(" world");
}

public static void main(String[] args) {
    StringBuilder message = new StringBuilder("Hello");
    addText(message);
    System.out.println(message); // Hello world
}
```



Both references reach the same builder. The method did not receive the caller's variable itself; it received a copy of the reference. That distinction matters in the next example.

8.3 Reassigning a parameter does not replace the caller's reference

```
public static void replaceBuilder(StringBuilder text) {
    text = new StringBuilder("New");
    text.append(" object");
}

public static void main(String[] args) {
    StringBuilder message = new StringBuilder("Original");
    replaceBuilder(message);
    System.out.println(message); // Original
}
```

The parameter `text` initially points to the caller's builder, but the assignment makes `text` refer to a different builder. The caller's `message` still refers to `Original`. This is why “objects are passed by reference” is not an accurate description of Java.

8.4 What about immutable Strings?

A String reference is copied just like any other object reference, but String objects are immutable. Concatenation creates a result String and assigns it to the local parameter, leaving the caller's reference unchanged.

```
public static void addExclamation(String text) {
    text += "!";
}

String greeting = "Hi";
addExclamation(greeting);
System.out.println(greeting); // Hi
```

9 Method Overloading

Overloading means declaring multiple methods with the same name but different parameter lists in the same class. Java chooses the applicable method using the arguments at compile time.

```
public static int multiply(int a, int b) {
    return a * b;
}

public static double multiply(double a, double b) {
    return a * b;
```

```
}

public static int multiply(int a, int b, int c) {
    return a * b * c;
}

System.out.println(multiply(2, 3)); // 6
System.out.println(multiply(2.5, 3.0)); // 7.5
System.out.println(multiply(2, 3, 4)); // 24
```

A method's name and parameter types are central to its **signature**. Changing only the return type does *not* create a valid overload:

```
// Not allowed in one class:
// public static int getValue() { return 1; }
// public static double getValue() { return 1.0; }
```

The caller cannot choose between these based solely on the variable receiving the result. Overloading also differs from **overriding**, which involves inheritance and will be introduced later.

10 Methods with Conditions and Loops

Methods can contain any ordinary Java control-flow statements. They become especially useful when a repeated calculation should be reusable.

10.1 Example: count the digits of a positive integer

```
public static int countDigits(int number) {
    if (number == 0) {
        return 1;
    }
    int count = 0;
    while (number > 0) {
        number /= 10;
        count++;
    }
    return count;
}
```

For `countDigits(4821)`, the variable `number` becomes 482, then 48, then 4, then 0. The counter becomes 1, 2, 3, and 4, so the method returns 4. This version assumes nonnegative input; negative numbers would need an additional design decision. The important idea is that the loop's local variables are created for each method call.

10.2 Example: determine whether a number is prime

```
public static boolean isPrime(int n) {
    if (n < 2) {
        return false;
    }
    for (int divisor = 2; divisor <= n / divisor; divisor++) {
```

```
        if (n % divisor == 0) {
            return false;
        }
    }
    return true;
}
```

The method immediately returns `false` for values below 2. Otherwise, it tests possible divisors only while `divisor <= n / divisor`, which avoids multiplying the divisor by itself and limits the search to the square-root region. If any divisor evenly divides `n`, we return `false` immediately. If the loop finishes, no divisor was found, so the method returns `true`. The method can be reused anywhere a boolean condition is needed:

```
if (isPrime(29)) {
    System.out.println("Prime number");
}
```

11 Multiple Calls Create Separate Invocations

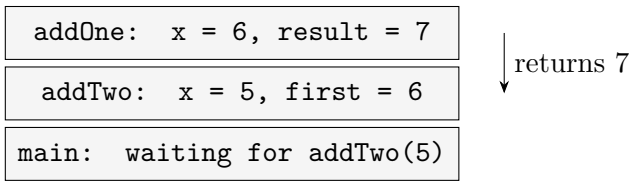
Calling a method twice does not reuse its previous local variables. Each call has its own invocation state. This is easiest to see when one method calls another method and then returns.

```
public static int addOne(int x) {
    int result = x + 1;
    return result;
}

public static int addTwo(int x) {
    int first = addOne(x);
    int second = addOne(first);
    return second;
}

System.out.println(addTwo(5)); // 7
```

The first `addOne` call receives 5 and returns 6. The second receives 6 and returns 7. Each call has its own `x` and `result`. A useful mental model is a **call stack**: active calls have separate stack frames containing their parameters, local variables, and the place to resume after returning. Actual JVM implementations may optimize storage, but this model correctly describes the behavior visible to the program.



12 Method Names, Documentation, and Good Design

A method name should describe its job. Java conventions use lower camel case: `calculateTotal`, `printReceipt`, and `isValidScore`. Names such as `doThing` or `stuff` hide the method's purpose. A

useful design rule is to give each method one clear responsibility and make its parameters and return value reflect that responsibility.

```
/**
 * Converts a score from 0 through 100 to a letter grade.
 * Returns '?' when the score is outside that range.
 */
public static char letterGrade(int score) {
    if (score < 0 || score > 100) {
        return '?';
    }
    if (score >= 90) return 'A';
    if (score >= 80) return 'B';
    if (score >= 70) return 'C';
    if (score >= 60) return 'D';
    return 'F';
}
```

This method returns a `char` instead of printing. The caller can decide whether to display it, store it, or use it in a condition. The documentation comment explains the expected input range and the behavior for invalid scores. This is part of a method's **contract**: what it expects and what it promises to produce.

13 A Complete Program Built from Several Methods

The following program combines methods with parameters, return values, conditions, and loops. Notice that `main` reads like a short outline of the program because the details are handled by helper methods.

```
public class ScoreReport {
    public static int sumScores(int a, int b, int c) {
        return a + b + c;
    }

    public static double average(int a, int b, int c) {
        return sumScores(a, b, c) / 3.0;
    }

    public static boolean passed(double average) {
        return average >= 60.0;
    }

    public static void printReport(int a, int b, int c) {
        double avg = average(a, b, c);
        System.out.println("Average: " + avg);
        if (passed(avg)) {
            System.out.println("Passed");
        } else {
            System.out.println("Not passed");
        }
    }

    public static void main(String[] args) {
        printReport(70, 80, 90);
    }
}
```

The call `printReport(70, 80, 90)` invokes `average`, which invokes `sumScores`. The sum is 240, the

average is 80.0, and `passed(80.0)` returns `true`. Therefore, the output is `Average: 80.0` followed by `Passed`. Each method has a clear purpose, and we can test `sumScores`, `average`, and `passed` separately.

14 Reading and Debugging Method Calls

When tracing a program with methods, identify the exact call, write down each argument value, then create a separate row or frame for the called method's parameters and local variables. Follow its statements until `return` or the end of a `void` method. Only then resume the caller. Do not assume that a local variable inside the called method changes a primitive variable in the caller.

```
public static int mystery(int x) {
    x += 3;
    return x * 2;
}

int x = 4;
int y = mystery(x);
System.out.println(x + " " + y); // 4 14
```

Step	Caller `x`	Parameter `x`	Result
Before call	4	—	—
Enter `mystery`	4	4	—
Execute `x += 3`	4	7	—
Execute `return x * 2`	4	7	14
Back in caller	4	out of scope	`y = 14`

14.1 Frequent syntax problems and what they mean

```
// Error: the parameter type is missing.
// public static int doubleIt(number) { return number * 2; }

// Error: return type int but no int is returned.
// public static int doubleIt(int n) { System.out.println(n * 2); }

// Error: cannot use a void call as a value.
// int x = printDouble(5);

// Error: calling an instance method without an object
// from a static context.
// increment(); // in Counter's static main
```

It is also possible to accidentally put a semicolon immediately after a method header, or place a helper method inside `main`. A normal method definition has a body enclosed in braces, and methods must be declared at the class level. For output tracing, pay attention to whether a method prints, returns, or both; those actions are not interchangeable.

15 Quick Reference: What to Look For in a Method

Concept	Meaning
Method definition	Declares the name, parameters, return type, and body
Method call	Runs the method using supplied arguments
Parameter	Local variable declared in a method header
Argument	Value or expression supplied at a call
`void`	No return value
`return value;`	End call and give a result to the caller
`static`	Method belongs to class, not a particular instance
Instance method	Method invoked on an object
Scope	Where a variable's name is accessible
Pass-by-value	Java copies each argument value into its parameter
Overloading	Same method name with different parameter lists